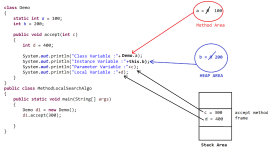


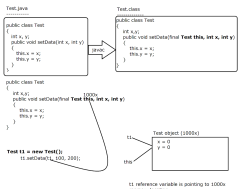
```
Variable Shadow in Java
If class level variables and method level variables are having exactly same name then method level
variables will hide them. But variables inside the method are constructor are class level, this concept is
known as variable shadow.
package com.test.variable.shadow;
class Student {
    private int roll = 100; //non static field
    private String name = "Rishi"; //non static field
    private String course = "Java"; //static field
    public void acceptInfo() {
        String name = "Rishi";
        String course = "Java";
        System.out.println("Roll number is: "+roll);
        System.out.println("Name is: "+name);
        System.out.println("Course is: "+course);
    }
}
public class VariableShadow {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.acceptInfo();
    }
}
Output is -> 100, Rishi, Java
* In case of variable shadowing, if we want to represent class level variables then we should use the
keyword.
a) for non static field -> we should use this keyword
b) for static field -> we should use class name
package com.test.variable.shadow;
class Student {
    private int roll = 100; //non static field
    private String name = "Rishi"; //non static field
    private String course = "Java"; //static field
    public void acceptInfo() {
        String name = "Rishi";
        String course = "Java";
        System.out.println("Roll number is: "+this.roll);
        System.out.println("Name is: "+this.name);
        System.out.println("Course is: "+Student.course);
    }
}
public class VariableShadow {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.acceptInfo();
    }
}
```

What is method local variable?

- Whenever any method, block or constructor is executed then first of all compiler will search the variable in the method, block or constructor (At Method Level).
- If variable declaration is not available at method level then compiler will search in the class (class level).



```
This Keyword in Java
Example for this keyword CustomerOrder.java
package com.test.variable.shadow;
class CustomerOrder {
    public static void main(String[] args) {
        Customer c1 = new Customer();
        c1.acceptInfo();
        c1.getCustomerData();
    }
}
* This keyword also refers to the current object i.e. by using this keyword we can access the
object properties (non static field) and object reference (non static method).
* Whenever non static field name and method level variables have both are exactly same then
compiler will search first, if we want to represent non static field then we should use the
keyword otherwise method level variable will hide non static field due to variable shadowing concept.
* method is not created at compile time but they are automatically generated with code.
Read this keyword as a first parameter to all the non static method and constructor.
```



```
* this keyword represents object hence it is a non static member so we cannot use this keyword
from static method, static context (Static Method, Static Block, Nested static class)
package com.test.this.keyword;
public class Customer {
    private int id;
    private String name;
    private String phone;
    public void acceptInfo() {
        String name = "Rishi";
        String phone = "98765";
        this.id = 100;
        this.name = "Rishi";
        this.phone = "98765";
    }
    public void getCustomerData() {
        System.out.println("ID is: "+this.id);
        System.out.println("Name is: "+this.name);
        System.out.println("Phone is: "+this.phone);
    }
}
package com.test.this.keyword;
public class CustomerDemo {
    public static void main(String[] args) {
        Customer c1 = new Customer();
        c1.acceptInfo();
        c1.getCustomerData();
    }
}
```